HOORVIA - OOP CONCEPTS CHART: PRINTABLE OBJECT-ORIENTED PROGRAMMING REFERENCE

Quick Reference Cheat Sheet · A4 Printable

Tech & Software · Sheet #1

Concept	What It Means	Example in Code	Why It Matters
Class	Blueprint defining attributes and behavior	class Car: ...	One definition builds many consistent objects
Object / Instance	A concrete thing built from a class	my_car = Car()	Classes only matter when instantiated
Constructor	Special method that builds an object	__init__(self, brand)	Guarantees every object starts in a valid state
Method	A function that belongs to an object	car.start()	Behavior lives with the data it acts on
Encapsulation	Bundling data together and hiding internals	self._mileage (private)	Stops other code from breaking your internals
Inheritance	Child class reuses the parent's features	class Truck(Car): ...	Reuse working code instead of rewriting it
Polymorphism	Same call, different behavior per object	car.start() vs truck.start()	One interface handles many object shapes
Abstraction	Show what it does, hide how it works	brake() without knowing hydraulics	Teams work on parts without knowing every detail
Interface / Abstract Class	A contract of methods a class must have	@abstractmethod	Forces consistent shapes across classes
Overriding vs Overloading	Replace or reuse a method signature	child.start() vs start(x) / start()	Specialize inherited behavior without touching the parent
Access Modifiers	Who can see what: public, protected, private	self._protected, self.__private	Controls the surface area others can misuse
Composition	Building objects from parts, not parents	self.engine = Engine()	'Has-a' beats deep 'is-a' inheritance chains

PRO TIP: Read real code: Open a popular library and label what you find - private fields are encapsulation, overridden methods are polymorphism.